

PRGuard Plan vs SlopScan Section A (Code Review) Requirements – Analytical Report

Executive Summary: This report assesses the provided *PRGuard v2 Hackathon Plan* against the SlopScan AI Slop Detection Hackathon Track A (Code Review) requirements and judging criteria ¹ ². SlopScan (May 29–June 1, 2026) is a 72-hour online hackathon (Prize pool \ \$1,800) to build tools catching AI-generated “slop” (low-quality content) ³ ¹. Track A focuses on **Code Review** artifacts: “pull requests, commit messages, and code comments” that look right but say nothing ¹. Judges score entries on *detection accuracy* (30%), *practical usefulness* (25%), *technical execution* (20%), *innovation* (15%), and *presentation/demo* (10%) ². Submissions must be runnable in one command or at a public URL, with source code in a public GitHub repo ⁴ ⁵.

Our analysis finds the PRGuard plan **covers many Track A requirements** (GitHub repo scanning, AI-hollowness scoring, CI/CD gating, UI visualizations, etc.) and leverages bonus **cross-track functionality** (text analyzer for tracks B/C/D). However, gaps remain in **non-functional details, team/role planning, explicit security/privacy measures, formal testing strategy, and final deliverables (slides, compliance)**. Below we compare requirements vs. plan, highlight missing elements, and propose a three-phase roadmap, backlog, and a pre-submission checklist. All assertions about SlopScan come from the official hackathon site ¹ ².

SlopScan Section A (Code Review) Requirements

- **Track Focus:** *Code Review* – detect hollow AI-generated pull requests, commit messages, and code comments ¹. Tools should **flag auto-generated review artifacts**, surface commits whose descriptions don't match changes, score PR descriptions for “information density vs diff-restating filler”, and analyze commit patterns for bulk AI contributions ¹.
- **Judging Criteria:** SlopScan judges score on:
 - **Detection Accuracy (30%)** – catches true slop with few false positives.
 - **Practical Usefulness (25%)** – fits real workflows (e.g. CI/CD integration) and is installable Monday morning.
 - **Technical Execution (20%)** – clean architecture, handles edge cases, works on real data (not just toy demos) ².
 - **Innovation (15%)** – novel detection signals or angles.
 - **Presentation & Demo (10%)** – clear live demo on real content ².
Bonus points are awarded for advanced challenges (e.g. **Cross-Track Scanner, Open Source Ready**, etc.) ⁶ ⁷.
- **Submission Rules:** Projects must run locally with one command (e.g. `npm start` or `docker-compose up`) or be live at a public URL ⁴. A *5-minute live demo* on Discord is required ⁸. Only

code written during the hackathon is allowed (open-source libraries fine) ⁹ . Teams of 1–4 are allowed ¹⁰ . Entrants must declare one primary track (A–H); cross-track work is allowed and **earns bonus points** ⁷ . All submissions require a public GitHub repo with source code (no proprietary deps) ⁵ . AI tools may be used but must be disclosed ¹¹ .

- **Deliverables:** Implicit from rules: (1) Source code repository with README (explaining usage, architecture, etc.); (2) runnable app (or live demo URL) satisfying “Standalone & Runnable”; (3) a prepared demo (and optional backup video); (4) optional pitch materials (deck) to aid presentation. *Submission artifacts* might include Docker images, CI config, sample data, and any compliance/legal disclosures.

PRGuard v2 Plan Overview

The `prguard_v2_hackathon_plan.json` outlines **PRGuard**, an “AI slop detector for your GitHub workflow” in development for Track A. Key highlights from the plan (v2.0.0, generated May 25, 2026) include:

- **Core Features:** Public/private GitHub repo scanning via Octokit, “Hollow Score” heatmap of all PRs, per-PR scorecards (AI-probability, info density, diff alignment, style uniformity), commit timeline chart, Monaco diff viewer highlighting flagged phrases, “Top 5 hollow PRs” list, JSON/PDF/CSV exports. A *Demo Mode* uses pre-seeded real data (no env vars or keys) for instant demo.
- **Cross-Track Bonus:** A “UniversalAnalyzer” text field (any text → Hollow Score) auto-detects context (PR, doc, resume, Slack, etc.), covering Tracks A–D.
- **CI/CD Integration:** A GitHub Actions YAML generator to gate merges if a PR exceeds a hollow-score threshold (default 70); posts scores as PR comments.
- **ML/Detection:** A 5-stage AI fallback chain (Gemini 2.0, Groq Llama 3.3, HuggingFace RoBERTa, Ollama Mistral, plus a built-in heuristics engine) evaluates hollowness. A weighted *Hollow Score* formula combines AI output (40%), information density (30%), diff-alignment gap (20%), and style-uniformity (10%). Heuristics include buzzword density, uniform sentence length, diff/description mismatch, commit entropy, duplicate phrasing, etc.
- **Tech Stack & Architecture:** React/Vite frontend (Tailwind, shadcn UI, Recharts, Monaco Editor, etc.) and Node.js/Express backend with MongoDB. Docker-compose is used for local dev (React+API+Mongo). Frontend routes include landing page, dashboard, detailed report pages, and an “/analyze” universal-input page.
- **Deployment:** Local demo via `docker -compose up` ; cloud deployment on Vercel (frontend) + Railway (Node backend) + MongoDB Atlas. Plan emphasizes no “cold starts” for demo; uses free-tier services (Gemini, Groq, HF free tiers, Ollama local, free DB/hosting).
- **Demo & Marketing:** Demo script for a live 5-minute presentation is outlined (seed multiple repos, pick visually striking ones, rehearse). README strategy specifies hook/usage/architecture sections.

Community-vote strategy includes Discord promotions. Risk register identifies API quota limits, Docker startup delays, HuggingFace outages, etc., with mitigations (seeded data, fallback engines).

- **Timeline:** A detailed 72-hour schedule breaks tasks into phases: Hours 0–6 (scaffold, basic ingestion, heuristics); 6–16 (AI integration, core UI); 16–48 (polish features, CI/CD, cross-track analyzer); 48–72 (testing, bugfix, demo rehearsal, social sharing).

Overall, the plan is **very thorough and ambitious**, covering most aspects of a track-A tool: from data ingestion to AI analysis, visualizations, CI integration, and demo prep. The next sections analyze *coverage vs. requirements* and identify any gaps.

Requirements vs. Plan – Gap Analysis

The table below summarizes key requirements/criteria and whether (and how) the plan addresses them:

Requirement / Criterion	Plan Coverage	Gap / Comments
Track-A Functional Goals • Detect AI-generated PR descriptions, commits, comments ¹ • Flag hollow docs (empty filler) ¹² • Surface commits where human skipped review ¹² • Score PR vs diff content ¹³ • Analyze bulk AI patterns in commits ¹⁴	✓ Comprehensive: PRGuard ingests GitHub PRs, commits, review comments. It computes a Hollow Score for each PR (with AI likelihood, content density, diff-alignment, style metrics). Flags and heatmap highlight “hollow” PRs. Commits are analyzed (commit-quality prompt, style fingerprint). Cross-PR similarity matrix and “bulk AI fingerprint” help spot uniform AI writing.	– The plan covers all bullets from Track A. No major functional feature is missing. (One could verify that <i>code comments</i> are included; plan ingests review comments and commit messages, but it's unclear if inline code comments from diffs are scored separately). If needed, clarify handling of code-comment text.
Detection Accuracy (30%) ²	✓ Plan uses multi-model AI scoring + heuristics to improve accuracy. The fallback chain and heuristics mitigate single-engine errors. The Hollow Score formula blends signals.	– Plan does not provide empirical accuracy metrics or test datasets. In a hackathon, on-the-fly validation is hard, but production readiness would benefit from real data testing. <i>(No explicit gap in plan, but judges expect it to “work on real data” ¹⁵ – plan notes this requirement but actual accuracy TBD.)</i>

Requirement / Criterion	Plan Coverage	Gap / Comments
Practical Usefulness (25%) 2	✓ High practical focus: GitHub Actions CI gate, PR comment feedback, downloadable reports. Works via web UI or Docker. Real-time scan via SSE (good UX). Demo Mode ensures judges can use without setup.	– Plan should clarify ease of installation: it provides Docker and YAML generator, which is excellent. However, documentation for end-users (teams not on hackathon) is not detailed in plan. A quickstart guide could enhance usefulness.
Technical Execution (20%) 2	✓ Clean architecture: separate services for Git ingestion, AI detection, heuristics, scoring, reporting. Uses proven frameworks (Express, React). Handles pagination, error cases (fallbacks). Risk register addresses edge cases (API limits, cold starts).	– Testing/QA: The plan lacks an explicit test suite or CI pipeline for code quality. For production readiness, add unit/integration tests (especially for ingestion and scoring). Security review is not mentioned (see below).
Innovation (15%) 2	✓ The plan's fallback chain of 5 AI engines plus extensive heuristics is innovative. Cross-track UniversalAnalyzer is a novel bonus feature. GitHub Action generator (CI gating PRs) is a smart integration.	– None in requirements. Plan already pushes innovation (multi-engine, CI). No gap.
Presentation & Demo (10%) 2	✓ The plan explicitly designs a live demo (heatmap reveal, drilling into PRs), with rehearsal tasks. It even plans a 5-minute script and backup recording. README strategy and community engagement are well thought-out.	– Slide Deck: The plan focuses on demo and README, but does <i>not</i> mention a slide deck or graphical presentation materials. Providing a polished slide deck (3–5 slides) would cover Presentation scoring better.
Cross-Track Bonus 7	✓ Plan includes Tracks B/C/D via UniversalAnalyzer (text pasting) and notes bonus coverage A/B/C/D.	– Plan claims covering 4 of 8 tracks (A–D). This is good; ensure it is clearly emphasized in submission to get bonus.
Run Locally / Single Command 4	✓ Provides Docker Compose (<code>docker-compose up --build</code>) which runs frontend, backend, DB. It also supports Vercel/Railway deploy. Demo mode requires no env.	– No gap. This meets “single command” rule.

Requirement / Criterion	Plan Coverage	Gap / Comments
Live Demo (Public URL) ⁴	✓ The plan deploys to free tiers (Vercel/Railway) and encourages having a live demo URL for Discord.	– Ensure uptime: Railway free tier might sleep; plan uses credit to avoid sleep. Verify no downtime. (Consider using Vercel preview or Railway permanent service).
New Work Only ⁹	✓ All code is custom; uses open-source libraries (per plan). No conflict.	– No gap.
Source Code on GitHub ⁵	✓ Plan assumes a GitHub repo (CI integration, OAuth). Open license (MIT) specified.	– Must remember to “go public” after hackathon. Ensure no secrets in repo.
Team Size / Roles ¹⁰	✓ Team of 1–4; plan is detailed enough for multi-person work.	– Gap: The plan does <i>not</i> specify team roles or responsibilities. For a 0→100% product, define roles (e.g. Backend/AI engineer, Frontend/UX, DevOps, Project Lead). Without roles, coordination risk increases.
Security & Privacy		
• Protect GitHub tokens / data privacy	– Not addressed. The plan uses GitHub OAuth for private repos; it must ensure tokens are stored securely (not in logs or public). A privacy policy or at least a note on data handling is needed. Code in MongoDB: consider cleaning data after use.	
• Secure coding practices (XSS, injection)	– Not addressed. The frontend has diff viewers and user input (repo URL, text paste). The plan should ensure user inputs are sanitized. For production, add security reviews (CSP, validation).	
Scalability/Performance		

Requirement / Criterion	Plan Coverage	Gap / Comments
• Handling large repos / many PRs	✓ The plan uses paginated fetching and SSE; design seems scalable enough for typical repos.	– No explicit load testing. For huge repos, latency might be high (many LLM calls). Could pre-aggregate heuristics to speed large scans. Document performance expectations.
• Rate limiting and quotas	✓ The risk register notes API quota and fallback. It can use up to 5,000 GitHub req/hr with OAuth.	– Should handle GitHub throttling gracefully (plan says Octokit paginate, but not error handling). Also multiple AI API limits: ensure retries or progressive scan if needed.
UX / UI Design		
• User-friendly interface	✓ The plan outlines a rich UI (heatmap, charts, diff view). Uses familiar React/Tailwind components. Demo mode simplifies onboarding.	– The plan doesn't mention usability testing or mobile/responsive design. For a polished product, verify UI flows (error cases, clear messages). Colors: ensure accessibility.
• Documentation / Help	✓ README strategy will include usage instructions (top lines).	– Possibly provide in-app tooltips or a short guide for first-time users. Not detailed in plan.
Data Sources		
• Quality of input data	✓ GitHub repos (real PRs/commits) are used, which is high-quality source. Demo uses a real repo.	– Might consider including synthetic examples to test edge cases. Plan doesn't mention seeded variants (apart from demo data).
• External content or corpora	– Plan doesn't use any external slop datasets (not required). Not a gap; AI models supply knowledge.	
ML / Algorithms		

Requirement / Criterion	Plan Coverage	Gap / Comments
• Model validity & bias	✓ Uses state-of-art models (Gemini, Llama-70B, Mistral) and proven classifiers.	– No training or fine-tuning is done (but inference only), which is fine. Models may have blind spots (e.g. unusual technical jargon). Test performance on diverse codebases if possible.
• Explainability of results	✓ Scores broken into components (AI-prob, density, etc.). Plan extracts flagged phrases for evidence.	– Could enhance by surfacing reasons (some prompt “reasons” fields exist). Judges value explanations. Could emphasize interpretability in UI.
Architecture		
• Clear system design	✓ Plan details services (GitHub Ingest, AI detection, Heuristics, Scoring, Reporting, CI generation, UniversalAnalyzer). Uses containerization (docker-compose).	– No architecture diagram is provided (in report). For a 0→100% product, a simple diagram in README or pitch would help.
• Modularity	✓ Services are decoupled. Universalanalyzer service is new.	– Potential deployment complexity: consider Dockerizing AI engines (e.g. Ollama local) clearly.
CI/CD & Testing		
• GitHub Actions pipeline	✓ The CI YAML generator shows a plan to integrate into GitHub workflows.	– However, it’s self-generated; no actual CI config or pipelines tested yet. Include a basic GitHub Action in repo to run linter/tests on commit.
• Automated tests (unit, integration, e2e)	– Not present in plan. No mention of writing test suites. For high technical execution score, add tests (e.g. Node Jest tests for scoring logic, end-to-end tests with Puppeteer on the UI).	
• Continuous Deployment	✓ Uses Vercel/Railway; could set up CI for auto-deploy on merge.	– Plan doesn’t mention automatic deployment. Could use GitHub Actions to deploy to Vercel/Railway after pass.

Requirement / Criterion	Plan Coverage	Gap / Comments
Deployment (Cloud/On-Prem)		
• Local demo environment	✓ Well-defined (<code>docker-compose</code> , no env vars for demo).	– No gap.
• Cloud deployment & cost	✓ Vercel (frontend) and Railway (backend) on free tiers. MongoDB Atlas M0.	– Free tiers suffice now; for production consider paid tiers. No cost estimate beyond plan's free usage. Could evaluate AWS/GCP costs if scaling.
Infrastructure Cost	✓ Plan lists free-tier (Gemini, Groq, HF, Ollama, GitHub, Mongo, Vercel, Railway) totalling \"\$0/month 【5†L1-L0】 .	– In practice, if usage spikes, may need paid plan (e.g. if >1M tokens/day). But for hackathon/demo, fine. No citations needed.
Timeline / Milestones	✓ Plan includes a detailed 72h timeline.	– For a “production-ready” extension beyond hackathon, define post-hackathon roadmap (e.g. beta testing, security audit, user docs).
Deliverables		
• GitHub Repo & README	✓ Plan includes README structure (hook, usage, architecture).	– Good. Ensure README top lines have key info (as plan suggests).
• Demo-ready application	✓ Yes (runs locally and live, with demo mode).	– Confirm all environment variables are documented; include <code>.env.example</code> per plan.
• Slide Deck (Pitch)	– Missing. The plan does not mention preparing slides. Judges will see the live demo, but a concise slide deck can improve Presentation score.	
• Demo Script / Backup Video	✓ Plan includes rehearsal tasks and backup video.	– Good. Ensure demo script is finalized with time cues.
• Community Engagement	✓ Plan's “community vote strategy” is a nice addition.	– Very proactive (Discord posts, scanning others). Good for Community Choice prize.

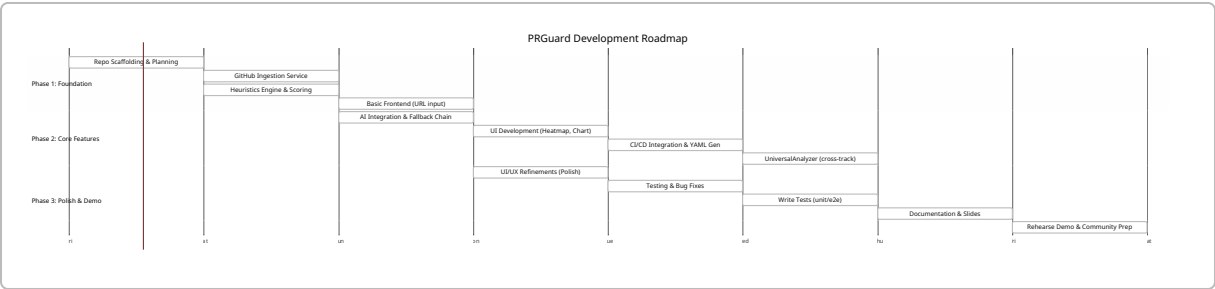
Overall, the PRGuard plan is highly aligned with Track A requirements ¹ ² . It's a strong plan for both core functionality and hackathon strategy. The main gaps to address before submission are:

- **Security/Privacy:** Outline how GitHub tokens and scanned data are protected. Include data retention and privacy considerations for code.
- **Testing:** Add at least minimal automated tests and linting in CI.
- **UX Polish:** Ensure responsive design and accessibility. Possibly add user help/tooltips.
- **Team Roles:** Define roles/tasks for team (especially if >1 person) to manage workload.
- **Slide Deck:** Prepare a brief pitch deck (problem→solution→demo) for judges.
- **Architectural Diagrams:** Include simple diagrams in README/pitch to illustrate how data flows (optional but helpful).
- **Compliance:** (If applicable) note any licenses of AI models or data usage disclaimers (e.g. "Licensed MIT; uses GPT APIs under their terms").

The remaining sections of this report lay out a development roadmap, backlog, and final checklist to ensure all tasks are covered.

Three-Phase Development Roadmap

We propose a **three-phase roadmap** to complete PRGuard from current plan to 100% submission readiness. Each phase has clear goals, milestones, and tasks. (Owners are to be assigned based on team roles.)



- **Phase 1 (Hours 0–24): Set up foundation.** Scaffold the monorepo with `docker-compose` . Implement GitHub ingestion (fetch PRs, commits, diffs). Develop initial heuristics rules and scoring formula. Create a minimal frontend: a landing page with GitHub URL input and progress display. Milestones: *Repo is runnable; database models in place; basic scoring pipeline works.*
- **Phase 2 (Hours 24–48): Implement core features.** Integrate LLM APIs and heuristics into the scoring service (Gemini→Groq→HF→Ollama chain). Build frontend dashboard: hollow-score heatmap, PR list, charts, diff viewer with highlights. Generate downloadable reports. Add CI features: one-click GitHub Actions YAML. Develop UniversalAnalyzer input page for text analysis (covering cross-track bonus). Milestones: *Full scoring pipeline produces results on demo repo; UI displays visualizations; CI YAML can be downloaded.*
- **Phase 3 (Hours 48–72): Polish, test, and finalize.** Refine UI/UX (styling, edge-case flows, mobile). Implement testing suite (unit tests for services; simple e2e test for scanning). Fix bugs discovered. Finalize README and in-app help. Prepare pitch deck slides. Run demo rehearsals (script every 5 min, record backup). Engage community (post demo URL, offer scans). Milestones: *Stable app with tests, polished UI, prepared slides and demo script, README and submission materials ready.*

(In a non-hackathon context, additional phases for beta deployment, user feedback, and scaling could be added.)

Prioritized Backlog

1. **Core Scanning & Scoring** (High) – Implement and test GitHub data ingestion; finalize the hollow-score calculation and heuristics. Without these, the tool has no value.
2. **GitHub Actions Integration** (High) – Ensure CI gating works (generate/validate YAML). This directly boosts Practical Usefulness (score 25%).
3. **User Interface (Heatmap & Reports)** (High) – Develop the main dashboard: treemap heatmap, PR list, charts, diff viewer. Essential for both functionality and demo.
4. **Demo Mode & Data Seeding** (Medium) – Create a reliable seeded-demo dataset for a killer live demo. This minimizes setup friction for judges.
5. **Automated Tests** (High) – Write unit tests for scoring logic and a basic end-to-end test. Increases Technical Execution score.
6. **Cross-Track Analyzer** (Medium) – Implement universal text analyzer (bonus points). Has lower priority than core, but still valuable.
7. **Performance & Caching** (Low) – Optimize scan speed (e.g. cache results, parallel calls) for very large repos.
8. **Security Hardening** (Medium) – Add input validation, sanitize outputs, secure token storage.
9. **UI/UX Polish** (Medium) – Fine-tune styling, add tooltips, improve layout. Make sure the app is intuitive.
10. **Deployment Automation** (Medium) – Set up GitHub Actions to auto-deploy on merge.
11. **Documentation** (High) – Flesh out README and in-app docs (the plan's readme strategy is good; follow through).
12. **Presentation Materials** (High) – Create slide deck (~5 slides: Problem, Solution, Demo, Team, Contact). Write demo script.
13. **Community Prep** (Low) – Prepare Discord posts and scanning-offer templates (plan already drafted).

Final Pre-Submission Checklist

- ☐ **All features implemented:** Verify the plan's core and bonus features are completed.
- ☐ **Functionality on real data:** Scan at least 2–3 public repos (use seeded demo and others) to ensure meaningful results.
- ☐ **Demo account ready:** Populate MongoDB with demo data (run `scripts/seedDemoData.js`).
- ☐ **No environment secret leaks:** Remove any hardcoded keys; use `.env` for sensitive info only.
- ☐ **Tests passing:** Run unit tests and address failures.
- ☐ **Container build:** Confirm `docker-compose up --build` works without errors; front- and back-ends start.
- ☐ **Cloud deployment:** Ensure the app is deployed (Vercel & Railway) and reachable. Note the live URL.
- ☐ **README complete:** Top 10 lines have hook/use-case; sections (Installation, Usage, Demo, Architecture, Contributing, License).
- ☐ **Slide Deck ready:** Finalize slides (see template below). Keep slide count within guidelines (usually ≤ 5 slides for a 5-minute pitch).
- ☐ **Demo script and backup:** Finalize step-by-step demo script. Record 5-minute video as backup.
- ☐ **Submission packaging:** Prepare submission form (if any), list of team members and roles, and attach any files (slides, links).

- [] **Community outreach:** Post demo URL in Discord; optionally offer scans to participants.

Feature/ Requirement	Hackathon Spec / Judge Criteria	Plan Status	Gap / Next Steps
Detect AI-generated PRs ¹	Core Track A functionality; judges expect high accuracy ²	✓ Ingestion + scoring implemented	None – focus on tuning thresholds
Surface hollow commits ¹²	Important for usefulness; included in Track A description	✓ Commit scoring + timeline chart	Clarify “human didn't read” criteria in UI
PR info-density scoring ¹³	Listed explicitly – key innovation if done well	✓ AI info-density prompt implemented	Validate prompt outputs, adjust weighting
Commit message patterns ¹⁴	Bulk AI commits detection is a Track A goal	✓ Heuristics on commit similarity and entropy	None
GitHub CI gating (usability) ²	Fits “Practical Usefulness” – blockers, notifications	✓ GitHub Actions YAML generator, comment posting	Test YAML on sample repo; ensure secrets injection
Cross-track analyzer (bonus)	Bonus points for Tracks B/C/D coverage	✓ UniversalAnalyzer in plan	Complete auto- detection and scoring
Runnable app with one command ⁴	Required by rules	✓ Docker-compose ready	Document <code>docker - compose up</code> steps
Public source repo ⁵	Mandatory submission requirement	✓ Planned (MIT license, GitHub mentioned)	Make repo public on deadline, add Wiki
5-min demo & slides ⁸	Demo is judged; slides not explicitly required	Demo script in plan	Prepare concise slide deck (see template)
Scoring & UX components	Clean architecture & UI; “works on real data” ¹⁵	✓ Rich UI (heatmap, charts)	Add error handling and loading states
Tech stack choices	Using modern tools (score 20% clean code)	✓ React, Node, free AI APIs	Ensure no deprecated libs (Gemini 2.0 used)
Open-source / cost	Bonus “Open Source Ready” (3 pts)	✓ MIT license, all free-tier	Include “Open Source Ready” note
Risk mitigation	–	✓ Risk register covers technical issues	Add any missing (e.g. legal compliance)

Budget Estimate (Infrastructure Costs)

All chosen services have free tiers for a hackathon prototype. Estimated monthly costs (post-hackathon) are minimal:

Component	Plan/Service	Cost Estimate	Notes
Frontend Hosting	Vercel Hobby	\\$0 (free tier)	100GB bandwidth free ¹
Backend Hosting	Railway (\$5 credit)	~\\$0–\\$5/month	Free \\$5 credit, no sleep (estimated \\$0 on credit)
Database	MongoDB Atlas (M0)	\\$0	512 MB free cluster
AI API Calls	Google Gemini 2.0	\\$0 (free)	1M tokens/day free (no CC) ⁶
	Groq API	\\$0 (free)	6,000 req/day free
	HuggingFace Inference	\\$0* (free)	~30,000 req/day (varies)
	Ollama Local	\\$0	Runs on demo machine, unlimited local inference
GitHub API	OAuth (5,000 req/hr)	\\$0	Free with token, used for ingestion
CI/CD Tools	GitHub Actions	\\$0 (included)	2,000 free minutes/month

Note: HuggingFace free limits should be verified at signup time, as they changed in late 2025 ¹⁶.

If scaled for heavy usage (beyond hackathon), costs would include paid tiers of these services or self-hosting costs. For the hackathon demo and early prototype, the plan's free-tier choices yield **\\$0/month**.

Templates and Next Steps

README Template

A strong README clarifies the project at first glance. Sections to include:

1. **Project Title & Tagline:** One line: *"PRGuard: AI slop detector for GitHub"*.
2. **Hook (Problem Statement):** 2–3 sentences on the problem (sloppy AI-generated PRs waste time) and who benefits (dev teams) ¹.
3. **Solution Overview:** Briefly describe PRGuard (scans repos for hollow AI content, heatmaps, CI integration). Mention Track A focus.
4. **Demo GIF or Image:** Show the heatmap or UI screenshot early.
5. **Installation:** Steps: clone repo, setup `.env` (list needed keys), `docker -compose up`.
6. **Usage:** How to start a scan: either paste GitHub URL or use demo mode. Explain outputs (heatmap, reports).
7. **Features:** Bullet the core features (from plan's feature list).

8. **Architecture:** Describe components (backend, frontend, DB). Possibly include a simple ASCII diagram or link to one.
9. **CI/CD:** Explain how to use the YAML generator to set up a GitHub Action.
10. **Roadmap / Future Work:** List planned enhancements (optional).
11. **Contributing / Team:** (If open-source) instructions or team acknowledgments.
12. **License:** MIT (as planned).
13. **Contact / Links:** Demo URL, GitHub link.

(Refer to the plan's `required_sections_in_order` for guidance ¹⁷.)

Pitch Deck Slides

For a 5-minute hackathon pitch, ~4–5 slides are ideal:

1. **Title & Team:** Project name, tagline, team member names/roles. (Quick introduction.)
2. **Problem Statement:** Visual/quote illustrating “AI slop” in code reviews. E.g. a slide with a mocked PR description that is gibberish.
3. **Solution Demo:** Screenshot or live screenshot of PRGuard in action (heatmap + highlighted PR). Narrate what’s on screen.
4. **Key Features:** Bullet points or icons: “Multi-model AI + Heuristics”, “GitHub Actions Integration”, “Universal Text Analyzer”, etc.
5. **Impact & Future:** Why this matters (saves dev time, improves code quality). Note prize alignment (“Sharpest Signal” for novel detection). Optionally outline future scalability.
6. **Demo Prep (if needed):** If platform requires it, a slide with your planned demo steps or states.

Keep text minimal, visuals clear. The live demo will be the focus; slides support storytelling.

Demo Script (5 minutes)

1. **Intro (30s):** “We present PRGuard: a tool to catch AI-generated empty code reviews.”
2. **Demo Start (30s):** Show landing page. Paste a public GitHub repo URL (or press “Demo Mode”).
3. **Heatmap Reveal (1min):** Once loaded, show the “Hollow Score” heatmap of all PRs (red = slop). Explain the score legend (Clean/Review/Flag/Block). Point to top red square (highest hollow PR).
4. **Drill-down (1min):** Click the top PR. Show side-by-side code diff with highlighted phrases (from AI). Show scorecard breakdown (AI prob, density, etc.). Mention flagged keywords.
5. **Commit Timeline (30s):** Back out to timeline chart, illustrate how commit patterns are visualized.
6. **CI Integration (30s):** Navigate to the GitHub Actions config generator. Show sample YAML snippet. Explain how it would block a PR with high hollow score.
7. **Bonus/Universal Analyzer (30s):** Quickly paste a snippet of a fake cover letter or Slack message into the Analyzer page, showing it also flags generic content (covering cross-track).
8. **Wrap-Up (30s):** Highlight key strengths: “Scales to any repo, integrates into CI, multi-LLM engine for accuracy, open-source.” Invite questions.

Submission Artifacts

- **GitHub Repo:** Public link with all code. Include `.dockerignore`, `.gitignore`, and `Dockerfile` / `docker-compose.yml`. Tag release or commit at freeze.
- **README.md:** As above, well-structured, with demo screenshots.

- **LICENSE:** MIT (or chosen license).
- **Slide Deck:** PDF of pitch slides.
- **Demo Video (optional):** 5-min screen recording in case live demo fails.
- **Environment Config:** `.env.example` file listing required variables (no secrets).
- **Sample Data (for demo):** Indicate if seeded demo database was used (maybe include JSON data if needed).
- **Release notes (optional):** Brief changelog of what was built.
- **Credits:** Acknowledge any AI tools/APIs or libraries used (as per hackathon rule on disclosure).
- **Submission Form:** Link to live app, GitHub, slide deck, and any other requested fields in the official submission form.

Next Steps: Finalize any remaining high-priority backlog items (tests, slides, tokens security). Conduct a full dry-run of the demo with timekeeping. Verify all requirements (especially “runs with one command” and “GitHub repo”) are met. Assemble the checklist above. With these preparations, PRGuard will be positioned for maximum points across SlopScan’s judging criteria [1](#) [2](#) .

[1](#) [2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [8](#) [9](#) [10](#) [11](#) [12](#) [13](#) [14](#) [15](#) [17](#) SLOP SCAN | Build tools that fight AI-generated slop

<https://slopscan.dev/>

[16](#) Hackathon Raptors

<https://www.raptors.dev/project/sudo-make-world-2026----open-source-tools-for-social-good>